APJ ABDUL KALAM TECHNOLOGICAL UNIVERSITY

KTU CST414 — DEEP LEARNING

MODULE I HANDWRITTEN LECTURE NOTES

B.Tech Computer Science and Engineering · Scheme 2019

Course Code: CST414	Credits: 3
Course Name: Deep Learning	Lecture Hours: 36
Faculty In-Charge: Jane Rose	Department: Computer Science & Engineering
Design: High-Contrast Printable Notebook	Font: Authentic Chalk & Kalam Ink

Prepared for students and faculty for classroom lectures, print study, and university examination preparation.

COURSE LECTURE INDEX & MAPPING

Day	Topic ID	Lecture Title	Module	CO	Date
#01	1.1	Introduction to Neural Networks - Single...	M1	CO1	18-Aug-2025
#02	1.2	Multi-Layer Perceptrons (MLPs) and Repre...	M1	CO1	20-Aug-2025
#03	1.3	Activation Functions, Risk Minimization ...	M1	CO1	22-Aug-2025
#04	1.4	Training MLPs with Backpropagation	M1	CO1	25-Aug-2025
#05	1.5	Illustration of the Backpropagation Algo...	M1	CO1	29-Aug-2025
#06	1.6	Practical Issues in Training: Overfittin...	M1	CO1	01-Sep-2025
#07	1.7	Convergence Difficulties, Local and Spur...	M1	CO1	10-Sep-2025

LECTURE #01 · MODULE 1 (Topic 1.1)

Date: 18-Aug-2025
Period: Period 2

Title: Introduction to Neural Networks – Single Layer Perceptrons

CO: CO1 Bloom's: Understand / Apply

Prerequisites: Linear Algebra (vectors, dot products), Basic Calculus, Python basics
Key Terminology: Artificial Neuron, Threshold Logic Unit (TLU), Perceptron Learning Rule, Linear Separability, Hyperplane Decision Boundary, Bias Weight

Introduction

Artificial Neural Networks (ANNs) are computational models inspired by the human biological nervous system. The fundamental computational unit is the artificial neuron, first conceptualized as the McCulloch-Pitts neuron (1943) and later modernized into the trainable Perceptron by Frank Rosenblatt (1958).

Mathematical Model

A Single Layer Perceptron receives an n-dimensional input vector x = [x1, x2, ..., xn]^T and computes the net input z (pre-activation) as the inner product with weight vector w = [w1, w2, ..., wn]^T plus bias b: z = sum_{i=1}^n (w_i * x_i) + b = w^T * x + b The output y is determined by a step activation function: y = f(z) = { +1 if z >= 0 (or 1 in unipolar) { 0 if z < 0 (or -1 in bipolar) Geometrically, w^T * x + b = 0 defines an (n-1)-dimensional hyperplane that partitions the input space into two decision regions.

Perceptron Learning Algorithm

Perceptron Learning Algorithm (PLA): 1. Initialize weights w and bias b to 0 or small random values. 2. For each training sample (x_k, d_k), where d_k is the target label: a. Compute predicted output: y_k = f(w^T * x_k + b) b. Compute error: e_k = d_k - y_k c. Update parameters using learning rate eta (0 < eta <= 1): w <- w + eta * (d_k - y_k) * x_k b <- b + eta * (d_k - y_k) 3. Repeat until all training patterns are classified correctly (zero error). Convergence Theorem (Novikoff, 1962): If the training data is linearly separable with margin gamma and bounded by ||x|| <= R, the PLA is guaranteed to converge in at most (R/gamma)^2 update steps.

Linear Separability And Limitation

Minsky & Papert (1969) proved that single-layer perceptrons can only classify linearly separable patterns. For non-linearly separable problems like the XOR function, no single linear decision boundary exists that can separate the classes, necessitating Multi-Layer Perceptrons.

Exam Important Points

- Derivation and geometric interpretation of the hyperplane decision boundary w^T * x + b = 0.
- Step-by-step weight updates using the Perceptron Learning Rule.
- Proof of linear separability for AND/OR and inability to solve XOR.
- Novikoff's Perceptron Convergence Theorem condition and upper bound on iterations.

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): Manual Output Calculation of Single Neuron

Statement: Given a 3-dimensional input vector x = [2, 2, 1]^T connected to a single neuron with weight vector w = [0.3, -0.4, 0.5]^T and bias b = -0.1. Calculate the net input (pre-activation) z and the final output y using (a) a unipolar step function and (b) a binary sigmoid activation function sigma(z) = 1 / (1 + e^(-z)).

Given: x = [2, 2, 1]^T, w = [0.3, -0.4, 0.5]^T, b = -0.1

Governing Formula: z = sum(w_i * x_i) + b; y_step = 1 if z >= 0 else 0; y_sigmoid = 1 / (1 + exp(-z))

Step-by-Step Calculation:

Step 1: Compute weighted sum: sum = (0.3 * 2) + (-0.4 * 2) + (0.5 * 1) = 0.6 - 0.8 + 0.5 = 0.3.

Step 2: Add bias: z = sum + b = 0.3 + (-0.1) = 0.2.

Step 3: Unipolar step function: Since z = 0.2 >= 0, y_step = 1.

Step 4: Binary sigmoid function: y_sigmoid = 1 / (1 + e^(-0.2)) = 1 / (1 + 0.81873) = 1 / 1.81873 = 0.5500.

[FINAL EVALUATED ANSWER: Net input z = 0.2, Step output = 1, Sigmoid output = 0.5500]

Pedagogical Note: The positive pre-activation indicates that the input vector lies on the positive side of the separating hyperplane. The sigmoid yields an activated confidence probability of 55%.

Problem 2 (Advanced (KTU Exam Level)): Perceptron Learning Rule Step-by-Step Weight Update

Statement: Design a single-layer perceptron to compute the 2-input NAND function. Initial weights are w1 = 0.0, w2 = 0.0 with bias b = 0.0. The learning rate is eta = 0.5. Perform the first epoch of training using the Perceptron Learning Algorithm for the sequential training samples: (0,0)->1, (0,1)->1, (1,0)->1, (1,1)->0. Show all intermediate calculations.

Given: Initial w = [0.0, 0.0], b = 0.0, eta = 0.5. Activation: step function (y=1 if z>=0 else 0). Samples: x1=(0,0), d1=1; x2=(0,1), d2=1; x3=(1,0), d3=1; x4=(1,1), d4=0.

Governing Formula: z = w1*x1 + w2*x2 + b; y = 1 if z >= 0 else 0; delta_w = eta * (d - y) * x; delta_b = eta * (d - y)

Step-by-Step Calculation:

Sample 1: x=(0,0), d=1. z = 0*0 + 0*0 + 0 = 0. Pred y = 1. Error = 1 - 1 = 0. No update. w=[0.0, 0.0], b=0.0.

Sample 2: x=(0,1), d=1. z = 0*0 + 0*1 + 0 = 0. Pred y = 1. Error = 1 - 1 = 0. No update. w=[0.0, 0.0], b=0.0.

Sample 3: x=(1,0), d=1. z = 0*1 + 0*0 + 0 = 0. Pred y = 1. Error = 1 - 1 = 0. No update. w=[0.0, 0.0], b=0.0.

Sample 4: x=(1,1), d=0. z = 0*1 + 0*1 + 0 = 0. Pred y = 1. Error = 0 - 1 = -1 (MISCLASSIFIED!).

Update on Sample 4: delta_w1 = 0.5 * (-1) * 1 = -0.5; delta_w2 = 0.5 * (-1) * 1 = -0.5; delta_b = 0.5 * (-1) = -0.5.

New weights after Epoch 1: w1 = -0.5, w2 = -0.5, b = -0.5.

[FINAL EVALUATED ANSWER: Weights after Epoch 1: w1 = -0.5, w2 = -0.5, b = -0.5.]

Pedagogical Note: The perceptron adjusted its hyperplane to penalize the false positive at (1,1). Further epochs will refine bias and weights until the decision boundary separates (1,1) from the remaining vertices.

Real-World Industry Case (Smart Manufacturing & Automated Quality Control): Industrial Defect Screening via Linear Separator

Pipeline Workflow: High-speed camera captures manufactured ceramic tiles -> Edge detection extracts contrast ratio (x1) and surface roughness variance (x2) -> Calibrated single-layer threshold neuron performs millisecond binary pass/fail classification.

Industrial Impact & Benefit: Enables ultra-low-latency (sub-millisecond) hardware defect filtering on microcontrollers without requiring deep GPU hardware.

Key Oral Viva Voce Checkpoints

Q1: What is the biological analogue of weights in an artificial neuron?

Ans: Synaptic connection strength (synapse efficacy).

Q2: Define linear separability in n-dimensional Euclidean space.

Ans: Two sets of points are linearly separable if there exists an (n-1)-dimensional hyperplane that cleanly divides the space with all points of one set on one side and the other set on the opposite side.

Q3: Why can a single-layer perceptron not solve the XOR problem?

Ans: Because the XOR truth table produces non-linearly separable patterns in 2D space where true and false instances lie on alternating diagonal vertices.

Q4: What role does the bias term play in a neuron?

Ans: The bias allows the decision hyperplane to shift away from the coordinate origin, enabling flexible positioning of the decision threshold.

Q5: State Novikoff's Perceptron Convergence Theorem.

Ans: If training samples are linearly separable with margin gamma and bounded norm R, the PLA will terminate with a separating solution in at most (R/gamma)^2 weight updates.

LECTURE #02 · MODULE 1 (Topic 1.2)

Date: 20-Aug-2025
Period: Period 4

Title: Multi-Layer Perceptrons (MLPs) and Representation Power of MLPs

CO: CO1 Bloom's: Understand / Analyze

Prerequisites: Single Layer Perceptron, Matrix multiplication, Linear Separability
Key Terminology: Input Layer, Hidden Layer, Output Layer, Universal Approximation Theorem, Feature Space Warping, Feedforward Architecture

Introduction

A Multi-Layer Perceptron (MLP) is a class of feedforward artificial neural network consisting of at least three layers of nodes: an input layer, one or more hidden layers, and an output layer. Except for input nodes, each node is a neuron that uses a non-linear activation function.

Architecture And Forward Pass

For an MLP with L layers: Layer 0: Input vector a^(0) = x For layer l = 1, 2, ..., L: z^(l) = W^(l) * a^(l-1) + b^(l) a^(l) = sigma(z^(l)) where W^(l) is an (n_l x n_{l-1}) weight matrix, b^(l) is an (n_l x 1) bias vector, and sigma is an element-wise non-linear activation function. The final network prediction is y_hat = a^(L).

Representation Power And Xor

How MLPs resolve the XOR problem: By mapping original 2D input space (x1, x2) through a hidden layer with 2 neurons (h1, h2), the input space is transformed into a new feature space where the points become linearly separable. Specifically: h1 = ReLU(x1 + x2 - 0.5) (acts as an OR-like detector) h2 = ReLU(x1 + x2 - 1.5) (acts as an AND-like detector) Output y = h1 - 2 * h2 produces exactly the XOR truth table!

Universal Approximation Theorem

Cybenko (1989) and Hornik (1991) proved the Universal Approximation Theorem: A standard feedforward neural network with a single hidden layer containing a finite number of neurons with non-linear activation functions can approximate any continuous function f: R^n -> R^m on compact subsets of R^n to arbitrary precision epsilon > 0. Crucial Caveat: The theorem guarantees the EXISTENCE of such an approximation, but it does NOT guarantee that the required weights can be discovered by gradient-based learning algorithms, nor does it guarantee that a single hidden layer is computationally or parameter-efficient compared to deeper architectures.

Exam Important Points

- Full architectural diagram of MLP with matrix dimensions for weights and biases.
- Algebraic demonstration of XOR resolution using hidden representations.
- Formal statement and engineering implications of the Universal Approximation Theorem.
- Calculation of total parameters (weights + biases) across multi-layer networks.

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): Trainable Parameter Count in Multi-Layer Perceptron

Statement: An MLP has an input layer with 784 features, a first hidden layer with 256 neurons, a second hidden layer with 128 neurons, and an output layer with 10 classes. Assuming all layers are fully connected with individual biases, calculate: (a) Number of weight parameters between each layer, (b) Number of bias parameters in each layer, and (c) Total trainable parameters in the entire network.

Given: Layer 0 (Input) n0 = 784, Layer 1 n1 = 256, Layer 2 n2 = 128, Layer 3 (Output) n3 = 10.

Governing Formula: Weights between layer (l-1) and l: W_l = n_l * n_{l-1}; Biases in layer l: B_l = n_l; Total = sum(W_l + B_l)

Step-by-Step Calculation:

Layer 1 (784 -> 256): Weights = 784 * 256 = 200,704. Biases = 256.
Layer 2 (256 -> 128): Weights = 256 * 128 = 32,768. Biases = 128.
Layer 3 (128 -> 10): Weights = 128 * 10 = 1,280. Biases = 10.
Total Weights = 200,704 + 32,768 + 1,280 = 234,752.
Total Biases = 256 + 128 + 10 = 394.
Total Trainable Parameters = 234,752 + 394 = 235,146.

[FINAL EVALUATED ANSWER: Total Trainable Parameters = 235,146]

Pedagogical Note: The majority of parameters (~85.3%) reside in the initial fully connected transition from input to the first hidden layer.

Problem 2 (Advanced (KTU Exam Level)): Feedforward Output Calculation of 2-Hidden-Layer MLP

Statement: Consider a 2-2-1 MLP with input x = [1.0, -1.0]^T. Hidden Layer 1 has weight matrix W^(1) = [[0.5, -0.2], [0.8, 0.1]] and bias b^(1) = [0.1, -0.2]^T with ReLU activation. Output layer has weight vector W^(2) = [0.7, -0.4] and bias b^(2) = 0.3 with Sigmoid activation. Calculate the step-by-step activations and the final network output.

Given: x = [1.0, -1.0]^T, W^(1) = [[0.5, -0.2], [0.8, 0.1]], b^(1) = [0.1, -0.2]^T, W^(2) = [0.7, -0.4], b^(2) = 0.3

Governing Formula: z^(1) = W^(1)*x + b^(1); a^(1) = max(0, z^(1)); z^(2) = W^(2)*a^(1) + b^(2); y = 1 / (1 + exp(-z^(2)))

Step-by-Step Calculation:

Step 1: Compute z1^(1) = (0.5 * 1.0) + (-0.2 * -1.0) + 0.1 = 0.5 + 0.2 + 0.1 = 0.8.
Step 2: Compute z2^(1) = (0.8 * 1.0) + (0.1 * -1.0) + (-0.2) = 0.8 - 0.1 - 0.2 = 0.5.
Step 3: Apply ReLU: a1^(1) = max(0, 0.8) = 0.8; a2^(1) = max(0, 0.5) = 0.5.
Step 4: Compute z^(2) = (0.7 * 0.8) + (-0.4 * 0.5) + 0.3 = 0.56 - 0.20 + 0.3 = 0.66.
Step 5: Apply Sigmoid: y = 1 / (1 + e^(-0.66)) = 1 / (1 + 0.51685) = 1 / 1.51685 = 0.6593.

[FINAL EVALUATED ANSWER: Hidden activations a^(1) = [0.8, 0.5]^T, Final Output y = 0.6593]

Pedagogical Note: Both hidden neurons are active (positive pre-activations), allowing information to propagate forward and yielding a positive prediction confidence of 65.93%.

Real-World Industry Case (Fintech & Credit Risk Assessment): Retail Loan Default Prediction Engine

Pipeline Workflow: Customer credit bureau features (debt-to-income, credit score, delinquencies, monthly inquiries) fed into a 4-layer MLP -> Hidden layers synthesize non-linear interaction patterns -> Output sigmoid neuron outputs probability of default within 90 days.

Industrial Impact & Benefit: Detects complex non-linear repayment risk signals that linear logistic regression models miss, cutting default rates by up to 18%.

Key Oral Viva Voce Checkpoints

Q1: What is the primary architectural difference between an SLP and an MLP?

Ans: An MLP contains one or more hidden layers between the input and output layers with non-linear activation functions.

Q2: Why is a non-linear activation function strictly required in hidden layers?

Ans: Without non-linear activations, a composition of linear layers collapses mathematically into a single linear transformation: W2*(W1*x + b1) + b2 = (W2*W1)*x + (W2*b1 + b2), removing the benefit of depth.

Q3: What does the Universal Approximation Theorem state?

Ans: A single hidden layer feedforward network with non-linear activations can approximate any continuous function on compact subsets of R^n to arbitrary accuracy, given sufficient hidden units.

Q4: If a shallow network can approximate any function, why do we use deep networks?

Ans: Shallow networks may require an exponentially large number of hidden neurons. Deep networks represent hierarchical compositional features with exponentially fewer parameters.

Q5: How many parameters are in a dense layer connecting 100 inputs to 50 outputs?

Ans: 100 * 50 = 5,000 weights + 50 biases = 5,050 trainable parameters.

LECTURE #03 · MODULE 1 (Topic 1.3)

Date: 22-Aug-2025
Period: Period 1

Title: Activation Functions, Risk Minimization and Loss Functions

CO: CO1 Bloom's: Understand / Apply

Prerequisites: Calculus (derivatives, partial derivatives), Probability basics
Key Terminology: Activation Function, Sigmoid, Hyperbolic Tangent (Tanh), Rectified Linear Unit (ReLU), Softmax, Empirical Risk Minimization, Cross-Entropy Loss, Negative Log-Likelihood

Activation Functions

1. Sigmoid (Logistic): sigma(z) = 1 / (1 + e^(-z)), Range: (0, 1) Derivative: sigma'(z) = sigma(z) * (1 - sigma(z)), Max derivative = 0.25 at z = 0. Drawbacks: Vanishing gradient for |z| >> 0; Not zero-centered (causes zig-zagging gradient updates). 2. Hyperbolic Tangent (Tanh): tanh(z) = (e^z - e^(-z)) / (e^z + e^(-z)), Range: (-1, 1) Derivative: tanh'(z) = 1 - tanh^2(z), Max derivative = 1.0 at z = 0. Advantage: Zero-centered output; faster convergence than sigmoid, but still saturates at extremes. 3. Rectified Linear Unit (ReLU): f(z) = max(0, z), Range: [0, infinity) Derivative: f'(z) = 1 if z > 0, else 0. Advantages: Computationally ultra-efficient, does not saturate in positive regime (mitigates vanishing gradients), induces sparse representations. Drawback: 'Dying ReLU' problem when neurons permanently output 0. 4. Softmax (Multiclass Output): softmax(z_i) = e^(z_i) / sum_{j=1}^K e^(z_j), Range: (0, 1), sum_{i=1}^K softmax(z_i) = 1. Converts raw logits into a valid categorical probability distribution.

Risk Minimization

Statistical Learning Theory defines expected risk R(theta) as the expected loss over the true data distribution p_data(x, y): R(theta) = E_{(x,y)~p_data} [L(f(x; theta), y)] Since p_data is unknown, Empirical Risk Minimization (ERM) replaces true expectation with sample average over N training examples: R_emp(theta) = (1 / N) * sum_{i=1}^N L(f(x_i; theta), y_i) Structural Risk Minimization (SRM) adds a regularization penalty Omega(theta) to control model capacity and avoid overfitting: R_struct(theta) = R_emp(theta) + lambda * Omega(theta)

Loss Functions

1. Mean Squared Error (Regression): L_MSE = (1 / 2) * (y - y_hat)^2 (or 1/N * sum) Derived from Maximum Likelihood Estimation under Gaussian noise assumption. 2. Binary Cross-Entropy (Binary Classification): L_BCE = - [y * log(y_hat) + (1 - y) * log(1 - y_hat)] 3. Categorical Cross-Entropy (Multiclass Classification): L_CCE = - sum_{k=1}^K y_k * log(y_hat_k) Pairing Softmax with Cross-Entropy prevents gradient saturation during backpropagation because derivative dL/dz = y_hat - y.

Exam Important Points

- Formulas and graphical sketches for Sigmoid, Tanh, ReLU, and Softmax along with their derivatives.
- Why ReLU accelerates deep network convergence compared to Sigmoid.
- Mathematical proof that d/dz [sigma(z)] = sigma(z) * (1 - sigma(z)).
- Derivation of the softmax derivative and why cross-entropy is the natural loss partner.

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): Softmax Probability and Cross-Entropy Loss Calculation

Statement: A deep neural network outputs logits z = [2.0, 1.0, -1.0]^T for a 3-class classification problem. The true class label is Class 1 (one-hot vector y = [1, 0, 0]^T). (a) Calculate the Softmax probability distribution, and (b) Calculate the Categorical Cross-Entropy loss.

Given: z = [2.0, 1.0, -1.0]^T, true label y = [1, 0, 0]^T

Governing Formula: softmax(z_i) = e^(z_i) / sum(e^z); L = - sum(y_i * ln(p_i))

Step-by-Step Calculation:

Step 1: Compute exponentials: e^2.0 = 7.3891, e^1.0 = 2.7183, e^(-1.0) = 0.3679.

Step 2: Compute denominator sum = 7.3891 + 2.7183 + 0.3679 = 10.4753.

Step 3: Compute probabilities: p1 = 7.3891 / 10.4753 = 0.7054 (70.54%) p2 = 2.7183 / 10.4753 = 0.2595 (25.95%) p3 = 0.3679 / 10.4753 = 0.0351 (3.51%).

Step 4: Compute Cross-Entropy Loss: L = - [1 * ln(0.7054) + 0 + 0] = - (-0.3490) = 0.3490.

[FINAL EVALUATED ANSWER: Softmax probabilities = [0.7054, 0.2595, 0.0351]^T; Loss L = 0.3490]

Pedagogical Note: The model assigns the highest probability (70.54%) to the ground truth class, resulting in a moderate cross-entropy loss of 0.3490.

Problem 2 (Advanced (KTU Exam Level)): Derivative Derivation & Vanishing Gradient Verification

Statement: Given a 4-layer network where each hidden neuron uses the Sigmoid activation function. At each layer, the pre-activation magnitude |z| >= 4.0. (a) Compute the maximum possible value of the activation derivative at each layer, and (b) Calculate the upper bound on the gradient propagation factor across the 4 layers: product_{l=1}^4 sigma'(z_l). Contrast this with ReLU.

Given: Number of layers L = 4, |z| >= 4.0, Sigmoid activation.

Governing Formula: sigma'(z) = sigma(z) * (1 - sigma(z)); For |z| >= 4, sigma(4) = 1 / (1 + e^(-4)) = 0.9820, sigma'(4) = 0.9820 * (1 - 0.9820) = 0.0177.

Step-by-Step Calculation:

Step 1: In the best case (|z| = 0), sigma'(0) = 0.25. But for |z| >= 4.0, sigma'(z) <= 0.0177.

Step 2: Calculate product of gradients across 4 layers under Best-Case (z=0): (0.25)^4 = 0.003906.

Step 3: Calculate product of gradients under Saturated Case (|z| >= 4): (0.0177)^4 = 9.80 x 10^(-8).

Step 4: Comparison with ReLU: For ReLU, for any positive input z > 0, f'(z) = 1.0. Thus, product = (1.0)^4 = 1.0.

[FINAL EVALUATED ANSWER: Saturated gradient product = 9.80 x 10^(-8) (vanishes to near zero). ReLU gradient product = 1.0 (retained).]

Pedagogical Note: This illustrates mathematically why Sigmoid networks suffer from catastrophic vanishing gradients in deep architectures, whereas ReLU preserves gradient magnitude.

Real-World Industry Case (Computer Vision & Edge Automotive Systems): Autonomous Vehicle Traffic Sign Recognition

Pipeline Workflow: Input camera frame -> Deep network extracts visual features -> Softmax layer outputs probability across 43 traffic sign categories (Speed limit, Stop, Yield) -> Cross-entropy loss trains system to minimize classification risk.

Industrial Impact & Benefit: Softmax probabilities allow the automotive decision control unit to establish confidence thresholds (e.g. only brake if Stop Sign confidence > 98%).

Key Oral Viva Voce Checkpoints

Q1: What is the maximum value of the derivative of the standard logistic sigmoid?

Ans: 0.25, occurring at z = 0.

Q2: Why is Tanh generally preferred over Sigmoid in hidden layers?

Ans: Tanh is zero-centered (range -1 to 1), which prevents systematic directional bias in weight updates across successive layers.

Q3: What is the 'Dying ReLU' problem?

Ans: When a neuron's pre-activation is consistently negative, its gradient becomes permanently 0, preventing the weights from ever updating again during training.

Q4: How does Leaky ReLU solve the dying ReLU issue?

Ans: By introducing a small positive slope (e.g., 0.01*z) for negative inputs so gradients never become completely zero.

Q5: Why is Softmax combined with Cross-Entropy instead of Mean Squared Error?

Ans: Cross-Entropy cancels out the exponential term in Softmax, yielding a clean linear error term (y_hat - y) that prevents gradient saturation during backpropagation.

LECTURE #04 · MODULE 1 (Topic 1.4)

Date: 25-Aug-2025
Period: Period 2

Title: Training MLPs with Backpropagation

CO: CO1 Bloom's: Understand / Apply

Prerequisites: Chain Rule of Multivariable Calculus, Vector and Matrix Calculus, Forward Propagation
Key Terminology: Backpropagation, Chain Rule, Error Sensitivity (Delta), Gradient Vector, Jacobian Matrix, Forward Pass, Backward Pass

Introduction

Backpropagation (Rumelhart, Hinton, Williams, 1986) is the foundational training algorithm for multi-layer neural networks. It efficiently computes the partial derivative of the scalar loss function L with respect to every weight and bias parameter in the network by applying the chain rule in reverse order.

Mathematical Derivation

Consider layer l with activation a^(l) = sigma(z^(l)), where z^(l) = W^(l) * a^(l-1) + b^(l). Define the Error Sensitivity (Delta) vector for layer l: delta^(l) = del L / del z^(l) 1. Fundamental Equation 1 (Output Layer Delta): delta^(L) = (del L / del a^(L)) elementwise_prod sigma'(z^(L)) For Cross-Entropy Loss with Softmax: delta^(L) = y_hat - y 2. Fundamental Equation 2 (Hidden Layer Delta Backpropagation): delta^(l) = [(W^(l+1))^T * delta^(l+1)] elementwise_prod sigma'(z^(l)) This equation propagates error signals backward from layer l+1 to layer l. 3. Fundamental Equation 3 (Rate of change of Loss with respect to Bias): del L / del b^(l) = delta^(l) 4. Fundamental Equation 4 (Rate of change of Loss with respect to Weights): del L / del W^(l) = delta^(l) * (a^(l-1))^T Parameter Update Rule: W^(l) <- W^(l) - eta * (del L / del W^(l)) b^(l) <- b^(l) - eta * (del L / del b^(l))

Computational Efficiency

Naive numerical gradient estimation (perturbing each weight by epsilon) requires O(P) forward passes, leading to O(P^2) time complexity for P parameters. Backpropagation evaluates all P gradients in a single backward pass with computational cost roughly equivalent to 2 forward passes: O(P) complexity!

Exam Important Points

- State and derive the four fundamental equations of backpropagation.
- Define the error vector delta^(l) and explain its physical significance.
- Explain why backpropagation runs backward rather than forward.
- Matrix dimensional consistency across all weight gradient equations.

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): Output Layer Error Delta Calculation

Statement: An output neuron with Sigmoid activation has pre-activation z = 1.5, giving output a = 0.8176. The ground truth target is y = 1.0. If the loss function is Squared Error L = 0.5 * (a - y)^2, calculate: (a) Error derivative dL/da, (b) Activation derivative da/dz, and (c) Error sensitivity delta.

Given: z = 1.5, a = 0.8176, y = 1.0, L = 0.5*(a - y)^2

Governing Formula: dL/da = a - y; da/dz = a*(1 - a); delta = (dL/da) * (da/dz)

Step-by-Step Calculation:

Step 1: dL/da = 0.8176 - 1.0 = -0.1824.

Step 2: da/dz = 0.8176 * (1.0 - 0.8176) = 0.8176 * 0.1824 = 0.1491.

Step 3: delta = (-0.1824) * 0.1491 = -0.02720.

[FINAL EVALUATED ANSWER: dL/da = -0.1824, da/dz = 0.1491, delta = -0.0272]

Pedagogical Note: The negative delta indicates that increasing the pre-activation z will decrease the loss, which guides the gradient update in the proper direction.

Problem 2 (Advanced (KTU Exam Level)): Hidden Layer Weight Gradient via Error Backpropagation

Statement: In a 2-layer network, hidden neuron h1 has activation a1 = 0.6. It is connected to output neuron y with weight w_out = 0.8. The output neuron has computed error delta_out = -0.15. The hidden neuron h1 receives an input x1 = 0.5 through weight w_in, and its activation function is Sigmoid. Calculate: (a) The backpropagated error delta_h1, and (b) The gradient dL/dw_in.

Given: a1 = 0.6, w_out = 0.8, delta_out = -0.15, x1 = 0.5, hidden activation = Sigmoid.

Governing Formula: delta_h = (w_out * delta_out) * a_h * (1 - a_h); dL/dw_in = delta_h * x1

Step-by-Step Calculation:

Step 1: Compute activation derivative of hidden neuron: sigma'(z1) = a1 * (1 - a1) = 0.6 * (1 - 0.6) = 0.24.

Step 2: Backpropagate error through w_out: backprop_sum = w_out * delta_out = 0.8 * (-0.15) = -0.12.

Step 3: Compute hidden delta: delta_h1 = backprop_sum * sigma'(z1) = -0.12 * 0.24 = -0.0288.

Step 4: Compute gradient with respect to input weight: dL/dw_in = delta_h1 * x1 = -0.0288 * 0.5 = -0.0144.

[FINAL EVALUATED ANSWER: Hidden delta_h1 = -0.0288, Gradient dL/dw_in = -0.0144]

Pedagogical Note: The negative gradient means increasing w_in will decrease the overall network error. The weight update rule will increment w_in by eta * 0.0144.

Real-World Industry Case (Digital Signal Processing & Real-time Acoustics): Acoustic Noise Cancellation in Aviation Communication

Pipeline Workflow: Cockpit ambient microphone picks up jet engine roar -> Filtered through deep MLP -> Output produces anti-phase acoustic cancellation wave -> Error microphone measures residual decibel leak -> Backpropagation updates weights at 100 Hz.

Industrial Impact & Benefit: Enables crystal-clear air traffic controller communication in high-decibel turbofan cockpit environments.

Key Oral Viva Voce Checkpoints

Q1: Who popularized the Backpropagation algorithm for neural networks in 1986?

Ans: David Rumelhart, Geoffrey Hinton, and Ronald Williams.

Q2: What is the primary mathematical principle underlying backpropagation?

Ans: The chain rule of multivariable calculus applied in reverse topological order.

Q3: What does the term delta^(l) represent?

Ans: The partial derivative of the scalar loss with respect to the pre-activation vector z of layer l: del L / del z^(l).

Q4: Why is caching intermediate activations during the forward pass necessary?

Ans: Because weight gradients depend directly on the input activation from the preceding layer: dL/dW^(l) = delta^(l) * (a^(l-1))^T.

Q5: What is the time complexity of backpropagation compared to finite difference numerical gradient calculation?

Ans: Backpropagation is O(P) with respect to parameter count P, whereas finite differences require O(P^2) or O(P) full forward passes.

LECTURE #05 · MODULE 1 (Topic 1.5)

Date: 29-Aug-2025
Period: Period 1

Title: Illustration of the Backpropagation Algorithm

CO: CO1 Bloom's: Apply / Evaluate

Prerequisites: Backpropagation Equations, Matrix Calculus, Day 4 concepts
Key Terminology: End-to-End Walkthrough, Forward Evaluation, Backward Sensitivity, Gradient Update, Loss Reduction Verification, Learning Step

Network Specification

Standard 2-2-1 Fully Connected MLP Architecture: Input vector: x = [x1, x2]^T = [0.05, 0.10]^T Target output: y = 0.01 (or target vector for 2 outputs) Initial weights from Input to Hidden layer: V = [[v11, v12], [v21, v22]] = [[0.15, 0.20], [0.25, 0.30]] Hidden biases: b1 = [0.35, 0.35]^T Weights from Hidden to Output layer: W = [w1, w2] = [0.40, 0.45] Output bias: b2 = 0.60 Activation function: Logistic Sigmoid everywhere: sigma(z) = 1 / (1 + e^(-z)) Loss Function: Squared Error L = 0.5 * (y_hat - y)^2 Learning rate: eta = 0.5

Step By Step Execution Phases

Phase 1: Forward Pass 1. Hidden Neuron 1: z_h1 = (0.15 * 0.05) + (0.20 * 0.10) + 0.35 = 0.0075 + 0.020 + 0.35 = 0.3775 a_h1 = sigma(0.3775) = 1 / (1 + e^(-0.3775)) = 0.59327 2. Hidden Neuron 2: z_h2 = (0.25 * 0.05) + (0.30 * 0.10) + 0.35 = 0.0125 + 0.030 + 0.35 = 0.3925 a_h2 = sigma(0.3925) = 1 / (1 + e^(-0.3925)) = 0.59688 3. Output Neuron: z_o = (0.40 * 0.59327) + (0.45 * 0.59688) + 0.60 = 0.23731 + 0.26860 + 0.60 = 1.10591 a_o = sigma(1.10591) = 1 / (1 + e^(-1.10591)) = 0.75137 4. Loss Calculation: L = 0.5 * (0.75137 - 0.01)^2 = 0.5 * (0.74137)^2 = 0.27481 Phase 2: Backward Pass 1. Output Error Delta: dL/da_o = a_o - y = 0.75137 - 0.01 = 0.74137 da_o/dz_o = a_o * (1 - a_o) = 0.75137 * (1 - 0.75137) = 0.18681 delta_o = (dL/da_o) * (da_o/dz_o) = 0.74137 * 0.18681 = 0.13849 2. Output Weight Gradients: dL/dw1 = delta_o * a_h1 = 0.13849 * 0.59327 = 0.08216 dL/dw2 = delta_o * a_h2 = 0.13849 * 0.59688 = 0.08266 dL/db2 = delta_o = 0.13849 3. Hidden Neuron Deltas: delta_h1 = (delta_o * w1) * a_h1 * (1 - a_h1) = (0.13849 * 0.40) * 0.59327 * (1 - 0.59327) = 0.05540 * 0.24130 = 0.01337 delta_h2 = (delta_o * w2) * a_h2 * (1 - a_h2) = (0.13849 * 0.45) * 0.59688 * (1 - 0.59688) = 0.06232 * 0.24061 = 0.01499 4. Input-to-Hidden Weight Gradients: dL/dv11 = delta_h1 * x1 = 0.01337 * 0.05 = 0.0006685 dL/dv12 = delta_h1 * x2 = 0.01337 * 0.10 = 0.001337 Phase 3: Parameter Updates (eta = 0.5) w1_new = 0.40 - (0.5 * 0.08216) = 0.35892 w2_new = 0.45 - (0.5 * 0.08266) = 0.40867 v11_new = 0.15 - (0.5 * 0.0006685) = 0.14967

Exam Important Points

- Full 14-mark KTU examination standard problem solving template.
- Clear notation distinguishing layer indices and neuron indices.
- Retention of at least 4 decimal places throughout all calculations.
- Demonstration of post-update forward pass verifying loss reduction.

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): Single Step Weight Update with Learning Rate

Statement: An output layer weight w has current value 0.45. The computed gradient of the loss function with respect to w is dL/dw = 0.08266. If the learning rate is eta = 0.5, calculate: (a) The parameter update step delta_w, and (b) The new updated weight w_new.

Given: w_current = 0.45, dL/dw = 0.08266, eta = 0.5

Governing Formula: delta_w = - eta * (dL/dw); w_new = w_current + delta_w

Step-by-Step Calculation:

Step 1: Compute parameter update step: delta_w = - 0.5 * 0.08266 = -0.04133.

Step 2: Compute updated weight: w_new = 0.45 + (-0.04133) = 0.40867.

[FINAL EVALUATED ANSWER: delta_w = -0.04133, w_new = 0.40867]

Pedagogical Note: The positive gradient signifies that increasing w increases loss; hence, the gradient descent step reduces w by 0.04133.

Problem 2 (Advanced (KTU Exam Level)): KTU Model Paper Q11: Parameter Update in MLP using ReLU

Statement: Update the parameters in the given MLP using gradient descent with learning rate eta = 0.5 and activation function as ReLU. Input is x = [1.0, 1.0]^T, target y = 1.0. Initial weights: Hidden layer V = [[0.1, 0.2], [0.1, 0.1]] with hidden bias b1 = [0.0, 0.0]^T. Output layer W = [0.1, 0.1] with output bias b2 = 0.0. Loss function is L = 0.5 * (y_hat - y)^2. Perform one full step of gradient descent.

Given: x = [1.0, 1.0]^T, y = 1.0, eta = 0.5, V = [[0.1, 0.2], [0.1, 0.1]], b1=[0,0]^T, W = [0.1, 0.1], b2 = 0.0. ReLU activation everywhere.

Governing Formula: z1 = V*x + b1; a1 = max(0, z1); zo = W*a1 + b2; y_hat = max(0, zo); dL/da = y_hat - y; delta_o = dL/da * (1 if zo > 0 else 0)

Step-by-Step Calculation:

Step 1: Hidden layer forward pass: z1_1 = 0.1*1.0 + 0.2*1.0 = 0.3 -> a1_1 = max(0, 0.3) = 0.3 z1_2 = 0.1*1.0 + 0.1*1.0 = 0.2 -> a1_2 = max(0, 0.2) = 0.2.

Step 2: Output layer forward pass: zo = 0.1*0.3 + 0.1*0.2 + 0.0 = 0.03 + 0.02 = 0.05. y_hat = max(0, 0.05) = 0.05.

Step 3: Loss calculation: L = 0.5 * (0.05 - 1.0)^2 = 0.5 * (-0.95)^2 = 0.45125.

Step 4: Output layer gradients: dL/dy_hat = 0.05 - 1.0 = -0.95. dy_hat/dzo = 1.0 (since zo = 0.05 > 0). delta_o = -0.95 * 1.0 = -0.95. dL/dW1 = delta_o * a1_1 = -0.95 * 0.3 = -0.285. dL/dW2 = delta_o * a1_2 = -0.95 * 0.2 = -0.190.

Step 5: Hidden layer gradients: Since z1_1, z1_2 > 0, ReLU derivatives are 1.0. delta_h1 = (delta_o * W1) * 1.0 = (-0.95 * 0.1) * 1.0 = -0.095. delta_h2 = (delta_o * W2) * 1.0 = (-0.95 * 0.1) * 1.0 = -0.095. dL/dV11 = delta_h1 * x1 = -0.095 * 1.0 = -0.095. dL/dV12 = delta_h1 * x2 = -0.095 * 1.0 = -0.095. dL/dV21 = delta_h2 * x1 = -0.095 * 1.0 = -0.095. dL/dV22 = delta_h2 * x2 = -0.095 * 1.0 = -0.095.

Step 6: Parameter Updates (eta = 0.5): W1_new = 0.1 - 0.5 * (-0.285) = 0.1 + 0.1425 = 0.2425. W2_new = 0.1 - 0.5 * (-0.190) = 0.1 + 0.0950 = 0.1950. V11_new = 0.1 - 0.5 * (-0.095) = 0.1 + 0.0475 = 0.1475. V12_new = 0.2 - 0.5 * (-0.095) = 0.2 + 0.0475 = 0.2475. V21_new = 0.1 - 0.5 * (-0.095) = 0.1 + 0.0475 = 0.1475. V22_new = 0.1 - 0.5 * (-0.095) = 0.1 + 0.0475 = 0.1475.

[FINAL EVALUATED ANSWER: Updated W = [0.2425, 0.1950], Updated V = [[0.1475, 0.2475], [0.1475, 0.1475]]]

Pedagogical Note: Because the model severely underpredicted the target (y_hat=0.05 vs y=1.0), all weight gradients are negative, leading to significant positive weight increments to boost future predictions.

Real-World Industry Case (Electric Vehicles & Battery Management Systems (BMS)): Real-Time Battery State-of-Charge (SoC) Calibration in EVs

Pipeline Workflow: BMS monitors cell voltage, discharge current, and ambient temperature -> 2-hidden-layer MLP predicts battery State-of-Charge -> At charging stations, precise open-circuit voltage calibration triggers single-step backpropagation on-board microchip.

Industrial Impact & Benefit: Maintains accurate driving range estimation over 10-year battery degradation curves without requiring cloud data transfer.

Key Oral Viva Voce Checkpoints

Q1: What is the primary benefit of working through backpropagation manually?

Ans: It solidifies understanding of chain rule mechanics, tensor dimensions, and why error signals scale across layers.

Q2: How does learning rate magnitude impact the weight update step?

Ans: A high learning rate creates large updates that can overshoot the minimum and diverge; a low rate causes slow, computationally prohibitive convergence.

Q3: What guarantees that a backpropagation gradient update reduces the loss?

Ans: For an infinitesimally small learning rate eta, the update moves in the negative gradient direction (direction of steepest descent), guaranteeing non-increasing loss on smooth surfaces.

Q4: What happens to ReLU gradients when pre-activation z is exactly 0?

Ans: The mathematical derivative is technically undefined, but in practical deep learning frameworks it is defined via sub-gradient as either 0.0 or 1.0 (typically 0.0).

Q5: Why are bias gradients simply equal to the layer delta?

Ans: Because the derivative of pre-activation z with respect to bias b is identically 1 (dz/db = 1).

LECTURE #06 · MODULE 1 (Topic 1.6)

Date: 01-Sep-2025
Period: Period 2

Title: Practical Issues in Training: Overfitting, Vanishing and Exploding Gradients

CO: CO1 Bloom's: Understand / Analyze

Prerequisites: Backpropagation Algorithm, Linear Algebra (eigenvalues), Optimization basics
Key Terminology: Overfitting, Generalization Gap, Vanishing Gradients, Exploding Gradients, Product of Jacobians, Gradient Clipping, Weight Initialization

Overfitting

Overfitting occurs when a neural network memorizes training data noise and idiosyncratic artifacts rather than learning generalizable underlying patterns. Key Indicators: - Training Loss continues decreasing monotonically toward zero. - Validation / Test Loss reaches an inflection point and begins increasing steadily. - Large generalization gap: Loss_val - Loss_train >> 0. Fundamental Causes: 1. Excessive model capacity (too many parameters relative to dataset size). 2. Insufficient training samples or unrepresentative data distribution. 3. Training for excessive epochs without early stopping.

Vanishing And Exploding Gradients

Consider a deep network with L layers. By the chain rule, the gradient of the loss with respect to activation at early layer 1 is: del L / del a^(1) = (del L / del a^(L)) * product_{k=1}^{L-1} (del a^(k+1) / del a^(k)) Each Jacobian matrix is: del a^(k+1) / del a^(k) = W^(k+1) * diag(sigma'(z^(k+1))) Case 1: Vanishing Gradients If the largest singular values of weight matrices ||W^(k)|| < 1 and saturating activations have sigma'(z) <= 0.25 (e.g. Sigmoid): || del L / del a^(1) || <= || del L / del a^(L) || * product_{k=1}^{L-1} (0.25 * ||W^(k)||) -> 0 as L increases. Gradients at early layers shrink exponentially, stalling learning in shallow representations. Case 2: Exploding Gradients If weight matrices have spectral radius > 1 (e.g. weights initialized too large): || del L / del a^(1) || grows exponentially as (lambda_max)^L -> infinity. Results in numerical overflow (NaN / Inf loss), unstable oscillations, and ruined weights.

Mitigation Strategies

1. Activation Choice: Replace Sigmoid/Tanh with non-saturating activations (ReLU, Leaky ReLU, ELU). 2. Proper Initialization: Xavier/Glorot (1990) for Tanh; He/Kaiming (2015) for ReLU. 3. Gradient Clipping: Rescale gradient vector g if ||g|| exceeds threshold c: g_clipped = g * (c / ||g||) if ||g|| > c else g. 4. Normalization: Batch Normalization (Ioffe & Szegedy, 2015) and Layer Normalization. 5. Architectural: Residual skip connections (He et al., 2015) creating identity gradient highways.

Exam Important Points

- Mathematical chain rule derivation of vanishing gradient across L layers.
- Comparison of training vs validation loss curves indicating overfitting vs underfitting.
- Formula and geometric interpretation of Gradient Clipping by norm.
- How spectral radius of weight matrices dictates gradient explosion or decay.

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): Gradient Clipping by Norm

Statement: During backpropagation of a deep network, the computed gradient vector across all parameters is g = [3.0, -4.0, 12.0]^T. The maximum permissible gradient clipping threshold is c = 5.0. (a) Calculate the L2 norm of the gradient vector ||g||_2, (b) Determine whether clipping is required, and (c) Calculate the clipped gradient vector g_clipped.

Given: g = [3.0, -4.0, 12.0]^T, clipping threshold c = 5.0

Governing Formula: ||g||_2 = sqrt(sum(g_i^2)); if ||g|| > c then g_clipped = g * (c / ||g||_2) else g

Step-by-Step Calculation:

Step 1: Compute L2 norm: ||g||_2 = sqrt(3^2 + (-4)^2 + 12^2) = sqrt(9 + 16 + 144) = sqrt(169) = 13.0.

Step 2: Check clipping condition: Since ||g||_2 = 13.0 > c = 5.0, gradient clipping MUST be applied.

Step 3: Compute scaling factor: factor = c / ||g||_2 = 5.0 / 13.0 = 0.384615.

Step 4: Compute clipped components: g1_clipped = 3.0 * (5/13) = 15/13 = 1.1538 g2_clipped = -4.0 * (5/13) = -20/13 = -1.5385 g3_clipped = 12.0 * (5/13) = 60/13 = 4.6154.

Step 5: Verify new norm: sqrt(1.1538^2 + (-1.5385)^2 + 4.6154^2) = sqrt(1.3312 + 2.3670 + 21.3019) = sqrt(25.0) = 5.0.

[FINAL EVALUATED ANSWER: ||g|| = 13.0, Clipping required. Clipped gradient = [1.1538, -1.5385, 4.6154]^T (Norm = 5.0)]

Pedagogical Note: Clipping rescales the gradient vector to have a maximum length of 5.0 while strictly preserving its original descent direction.

Problem 2 (Advanced (KTU Exam Level)): Mathematical Estimation of Vanishing Gradient Decay Rate

Statement: A deep feedforward network has 8 layers. Each layer uses the standard sigmoid activation function where average pre-activation yields sigma'(z) = 0.20. The weight matrices at each layer have average spectral norm ||W||_2 = 1.5. (a) Derive the contraction factor per layer, (b) Calculate the total gradient attenuation factor from layer 8 back to layer 1, and (c) If the output error gradient is ||dL/da8|| = 2.0, determine the gradient magnitude arriving at layer 1.

Given: L = 8 layers, sigma'(z) = 0.20, ||W||_2 = 1.5, initial ||dL/da8|| = 2.0.

Governing Formula: Per-layer contraction factor gamma = ||W||_2 * sigma'(z); Total attenuation = gamma^(L-1); Magnitude at layer 1 = ||dL/da8|| * gamma^(L-1)

Step-by-Step Calculation:

Step 1: Compute per-layer contraction factor: gamma = 1.5 * 0.20 = 0.30.

Step 2: Number of inter-layer transitions across 8 layers = L - 1 = 7.

Step 3: Total attenuation factor = (0.30)^7 = 0.0002187 (or 2.187 x 10^(-4)).

Step 4: Compute gradient magnitude at layer 1: ||dL/da1|| = 2.0 * (0.30)^7 = 2.0 * 0.0002187 = 0.0004374 (or 4.374 x 10^(-4)).

[FINAL EVALUATED ANSWER: Per-layer factor = 0.30, Total attenuation = 2.187 x 10^(-4), Gradient at layer 1 = 0.0004374]

Pedagogical Note: The gradient signal is attenuated by over 99.97% before reaching the first hidden layer, meaning early layer weights learn thousands of times slower than output weights.

Real-World Industry Case (Healthcare Informatics & Predictive Medicine): Clinical Mortality Prediction in Intensive Care Units (ICU)

Pipeline Workflow: High-frequency ICU patient telemetry (ECG, SpO2, blood pressure, arterial blood gases) fed into an 8-layer deep network -> Overfitting prevented using early stopping based on validation cohort -> Gradient clipping prevents erratic updates during sudden patient physiological anomalies.

Industrial Impact & Benefit: Maintains reliable 24-hour mortality risk scores without overfitting to sparse hospital training records.

Key Oral Viva Voce Checkpoints

Q1: What is the key indicator of overfitting when observing learning curves?

Ans: Training loss continues to decrease while validation loss begins to increase steadily.

Q2: Mathematically, what causes the vanishing gradient problem?

Ans: The repeated multiplication of Jacobian matrices with spectral norms < 1 and activation derivatives bounded below 1 (e.g. 0.25 for sigmoid) across multiple layers.

Q3: What is the primary danger of exploding gradients?

Ans: They cause parameter updates so massive that weights overflow to NaN or infinity, completely destabilizing the network.

Q4: How does gradient clipping by value differ from gradient clipping by norm?

Ans: Clipping by value truncates each individual gradient element independently between [-c, c], which changes the gradient vector direction. Clipping by norm rescales the entire vector by c / ||g||, preserving its direction.

Q5: Why does the ReLU activation function mitigate vanishing gradients?

Ans: For any positive input z > 0, the derivative of ReLU is exactly 1.0, preventing exponential attenuation during backpropagation.

LECTURE #07 · MODULE 1 (Topic 1.7)

Date: 10-Sep-2025
Period: Period 4

Title: Convergence Difficulties, Local and Spurious Optima, Computational Challenges

CO: CO1 Bloom's: Understand / Analyze

Prerequisites: Optimization Theory, Multivariable Calculus (Hessian matrix, Eigenvalues), Day 6 concepts
Key Terminology: Non-Convex Optimization, Saddle Point, Spurious Local Minima, Hessian Matrix, Ill-Conditioned Curvature, Plateau, Condition Number

Loss Surfaces In Deep Learning

Unlike linear models whose convex loss functions guarantee a unique global minimum, deep neural networks have highly non-convex loss landscapes in millions of dimensions. The landscape is characterized by: 1. Symmetries and Permutation Invariance: Permuting neurons within a hidden layer yields identical input-output mappings, resulting in a factorial number of equivalent local minima. 2. Spurious Local Minima: Suboptimal local basins whose loss is significantly worse than the global minimum. Research (Dauphin et al., 2014; Choromanska et al., 2015) indicates that in high-dimensional deep models, true local minima are rare and generally close to global minimum loss. 3. Saddle Points: Critical points where gradient grad L = 0, but the Hessian has both positive and negative eigenvalues. In high dimensions, saddle points are exponentially more prevalent than local minima!

Ill Conditioned Hessian

*The Hessian matrix H contains second-order partial derivatives: H_ij = del^2 L / (del w_i del w_j). The condition number kappa(H) = lambda_max / lambda_min measures curvature disparity: - If kappa(H) >> 1, the loss surface forms an elongated canyon (ravine) where curvature is steep along one direction and flat along another. - First-order Gradient Descent oscillates back and forth across the steep walls while making agonizingly slow progress along the base toward the minimum. Second-order Taylor expansion of loss around current point w_0: L(w) approx L(w_0) + (w - w_0)^T * grad L + (1/2) * (w - w_0)^T * H * (w - w_0) A gradient descent step w = w_0 - eta * grad L yields expected loss change: delta L approx - eta * ||grad L||^2 + (1/2) * eta^2 * (grad L)^T * H * (grad L) If (grad L)^T * H * (grad L) is very large, gradient descent can actually INCREASE loss unless eta is miniscule.*

Computational Challenges

1. Memory Bandwidth Limits: Forward activations must be retained in GPU VRAM for the backward pass, limiting maximum batch size and model depth. 2. Computational Scale: Training modern deep architectures requires petascale floating-point calculations. 3. Vanishing Gradient Highways in Ravines: Stalling optimization on flat saddle plateaus.

Exam Important Points

- *Sketch and differentiate: Local minimum, global minimum, saddle point, and plateau.*
- *Define the Hessian condition number and explain why ill-conditioning causes gradient descent oscillation.*
- *Explain why saddle points pose a greater threat than local minima in high-dimensional deep networks.*
- *Derivation of expected loss change using second-order Taylor expansion.*

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): Hessian Matrix Condition Number and Curvature Evaluation

Statement: At a critical point in weight space, the Hessian matrix of a neural network loss function is computed as H = [[200.0, 0.0], [0.0, 2.0]]. (a) Identify the eigenvalues of H, (b) Calculate the condition number kappa(H), and (c) Characterize the nature of the curvature along both principal axes.

Given: H = [[200.0, 0.0], [0.0, 2.0]]

Governing Formula: Eigenvalues are diagonal entries for diagonal matrix; kappa = lambda_max / lambda_min

Step-by-Step Calculation:

Step 1: Eigenvalues are lambda_1 = 200.0 and lambda_2 = 2.0.

Step 2: Both eigenvalues are positive (lambda > 0), confirming that the critical point is a local minimum.

Step 3: Calculate condition number: kappa(H) = lambda_max / lambda_min = 200.0 / 2.0 = 100.0.

[FINAL EVALUATED ANSWER: Eigenvalues = 200.0 and 2.0. Condition number kappa = 100.0 (Ill-conditioned).]

Pedagogical Note: Curvature along the first coordinate axis is 100 times steeper than along the second axis. Vanilla gradient descent will severely oscillate along axis 1 while progressing very slowly along axis 2.

Problem 2 (Advanced (KTU Exam Level)): Second-Order Taylor Series Step-Size Bound

Statement: The loss surface around current weights has gradient g = [4.0, 1.0]^T and Hessian H = [[10.0, 0.0], [0.0, 1.0]]. Using the second-order Taylor expansion delta L approx - eta * ||g||^2 + 0.5 * eta^2 * (g^T * H * g), (a) Calculate the optimal learning rate eta* that maximizes loss reduction, and (b) Compute the maximum learning rate eta_max beyond which the step causes the loss to INCREASE instead of decrease.

Given: g = [4.0, 1.0]^T, H = [[10.0, 0.0], [0.0, 1.0]]

Governing Formula: ||g||^2 = g^T * g; g^T * H * g = sum(g_i^2 * H_ii); eta* = ||g||^2 / (g^T * H * g); eta_max = 2 * eta*

Step-by-Step Calculation:

Step 1: Compute ||g||^2 = 4^2 + 1^2 = 16 + 1 = 17.0.

Step 2: Compute curvature term: g^T * H * g = (4^2 * 10.0) + (1^2 * 1.0) = (16 * 10) + 1 = 160 + 1 = 161.0.

Step 3: Quadratic loss change equation: delta L(eta) = - 17.0 * eta + 0.5 * 161.0 * eta^2 = - 17.0 * eta + 80.5 * eta^2.

Step 4: Differentiate with respect to eta and set to 0: d(delta L)/d(eta) = -17.0 + 161.0 * eta = 0 => eta* = 17.0 / 161.0 = 0.10559.

Step 5: Compute threshold for loss increase (delta L = 0): -17.0 * eta + 80.5 * eta^2 = 0 => eta * (-17.0 + 80.5 * eta) = 0 => eta_max = 17.0 / 80.5 = 2 * eta* = 0.21118.

[FINAL EVALUATED ANSWER: Optimal learning rate eta* = 0.1056; Maximum learning rate before divergence eta_max = 0.2112]

Pedagogical Note: If the engineer sets a learning rate eta > 0.2112, the quadratic curvature penalty dominates the linear descent term, causing the loss to explode instead of decrease.

Real-World Industry Case (Geophysics & Energy Exploration): Seismic Oil & Gas Subsurface Reservoir Inversion

Pipeline Workflow: Acoustic wave reflection sensors collect terabytes of subterranean echoes -> Deep neural acoustic inversion predicts subterranean rock density and porous oil pockets -> Highly ill-conditioned Hessian landscapes caused by varied rock strata require adaptive curvature optimization.

Industrial Impact & Benefit: Prevents million-dollar exploratory dry-well drilling by resolving deep geological formation boundaries.

Key Oral Viva Voce Checkpoints

Q1: What is a saddle point in a multivariable function?

Ans: A critical point where the gradient is zero, but the Hessian has both positive and negative eigenvalues, meaning it is a minimum along some directions and a maximum along others.

Q2: Why are saddle points more common than local minima in deep networks?

Ans: In an N-dimensional space, for a critical point to be a local minimum, all N eigenvalues of the Hessian must be simultaneously positive (probability ~ (1/2)^N for random signs). For large N, this probability vanishes, making saddle points exponentially more frequent.

Q3: What is the condition number of the Hessian matrix?

Ans: The ratio of its maximum eigenvalue to its minimum eigenvalue: kappa = lambda_max / lambda_min.

Q4: How does an ill-conditioned Hessian affect gradient descent?

Ans: It creates severe oscillations perpendicular to the ravine walls while causing agonizingly slow progress along the canyon floor.

Q5: What is a plateau in the loss surface?

Ans: A region where the loss surface is nearly flat with vanishingly small gradients across multiple dimensions, drastically slowing down training.
